

Memory Management

- ❑ Memory Management Strategies:

- Swapping
- Contiguous Memory Allocation
- Segmentation
- Paging

- ❑ Virtual Memory Management

- Demand Paging
- Copy-on-Write

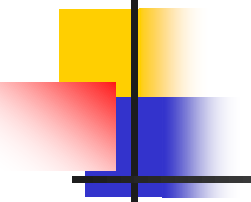
- ❑ Page Replacement

- Thrashing

- ❑ File System

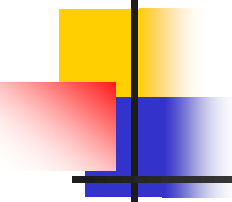
- File concept
- Access Methods

- ❑ Directory & Disk Structure



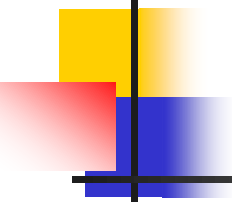
Background

- Program must be brought into memory and placed within a process for it to be run.
- To improve both the utilization of the CPU and the speed of its response to users, a general-purpose computer must keep several processes in memory.
- OS provides Many memory-management schemes to improve memory utilization.
- **Input queue or job queue** – collection of processes on the disk that are waiting to be brought into memory to run the program.
- OS has to perform various steps in order to execute the processes such as address and data binding.



Addresses Binding

- There are two types of addresses: logical address and physical address.
- **Logical address** is generated by CPU during a program execution. It is also called as virtual address.
- **Physical Address** identifies a physical location in a memory unit.
- Set of all logical addresses generated by CPU in reference to a program is referred as **Logical Address Space**.
- Set of all physical addresses mapped to the corresponding logical addresses is referred as **Physical Address Space**.
- **Address binding** is the process of mapping from one address space to another address space.



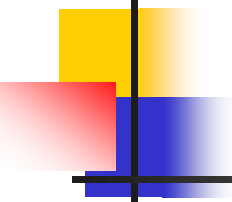
Address Binding can happen at three different stages:

1. Compile-time Address Binding:

- It will be done before loading the program into memory.
- Compiler is responsible for performing address binding.
- The compiler requires interacts with an OS memory manager to perform compile-time address binding.

2. Load time Address Binding:

- It will be done after loading the program into memory.
- This type of address binding will be done by the OS memory manager i.e loader.

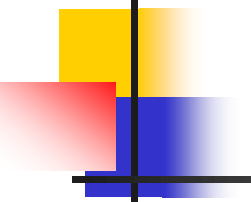


3. Execution time / Dynamic Address Binding:

- Binding done by the processor at the time of program execution.
- Need hardware support for address maps.
- The program will be kept on changing the locations in memory until the time of program execution.

➤ MMU(Memory Management Unit)

- The run time mapping between Virtual address and Physical Address is done by hardware device known as MMU.
- In memory management, Operating System will handle the processes and moves the processes between disk and memory for execution.
- It keeps the track of available and used memory.



Dynamic Loading

- In dynamic loading, a routine is not loaded until it is called.
- All routines are kept on disk in a relocatable load format.
- The main program is loaded into memory and is executed.
- When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded.
- If it has not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change.
- Then control is passed to the newly loaded routine.
- The advantage of dynamic loading is to provide better memory-space utilization.

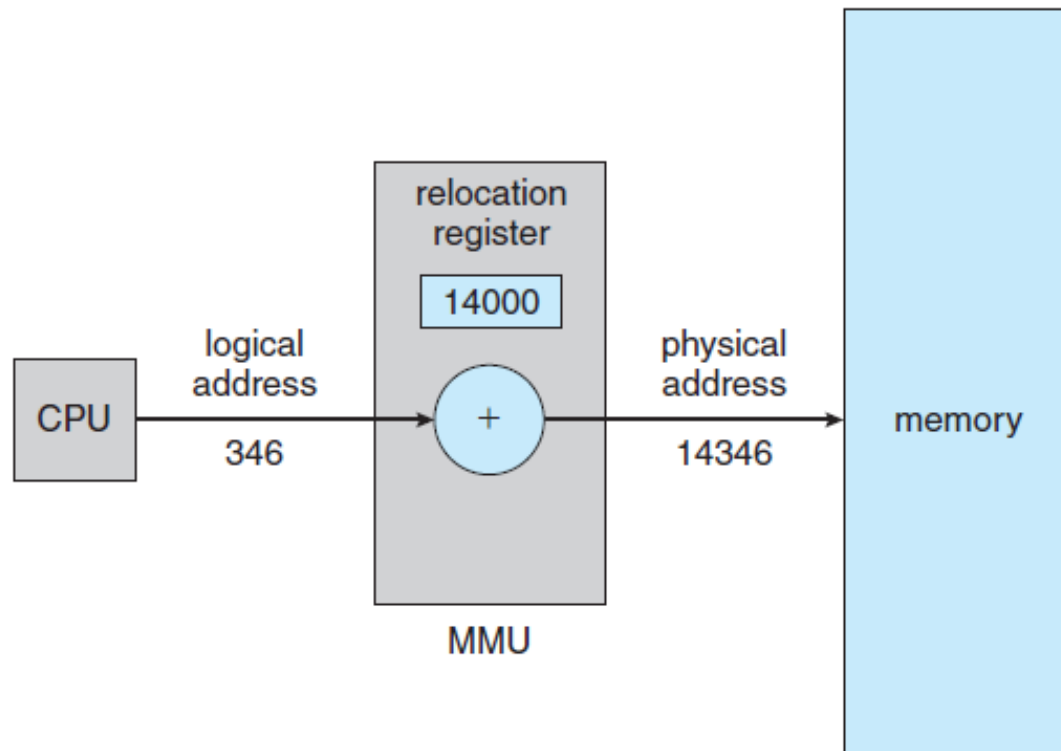
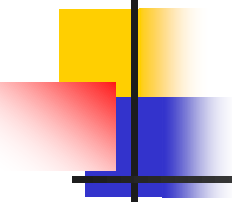


Figure 8.4 Dynamic relocation using a relocation register.

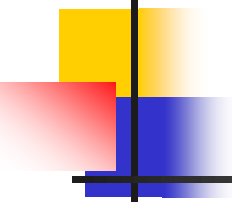


Overlays in Memory Management

- Overlay memory management technique allows multiple programs to be loaded into memory simultaneously, but only a portion of each program is resident in memory at any given time.
- The technique swaps different parts of the programs in and out of memory as required.
- It is commonly used in situations where the memory requirements of the programs exceed the available physical memory.

Advantages

1. Improved Memory Utilization
2. Reduced Memory Fragmentation
3. Reduced Memory Access Time
4. Improved Performance



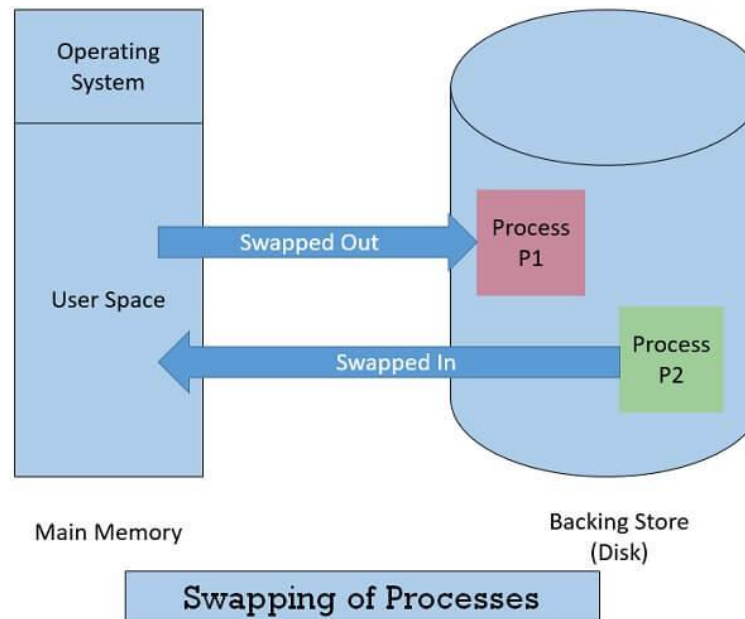
Disadvantages

1. Increased Complexity
2. Limited Address Space
3. Increased Overhead

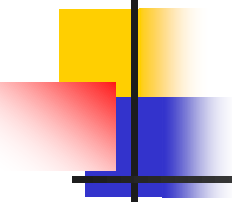
➤ **Swapping**

- Swapping is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes.
- A process will be moved to temporarily out of memory to a backing store and then brought back into memory for continued execution.

- Swapping makes it possible for the total physical address space of all processes to exceed the real physical memory of the system, thus increasing the degree of multiprogramming in a system.

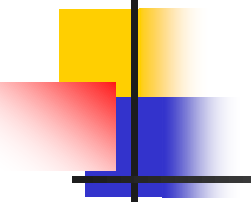


- The backing store is a fast disk which must be large enough to accommodate copies of all memory images for all users.

- 
-
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped.

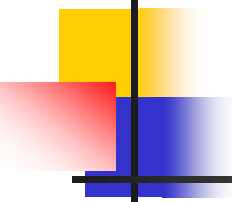
Benefits of Swapping:

- It offers a higher degree of multiprogramming.
- Allows dynamic relocation.
- It helps to get better utilization of memory



Contiguous Memory Allocation

- The main memory must accommodate both the operating system and the various user processes.
- Therefore we need to allocate main memory in the most efficient way possible.
- The memory is usually divided into two partitions: one for the resident operating system and one for the user processes.
- In contiguous memory allocation, each process is contained in a single section of memory that is contiguous to the section containing the next process.
- Simplest methods for allocating memory is to divide memory into several fixed-sized partitions.
- Each partition may contain exactly one process. Thus, the degree of multiprogramming depends on number of partitions.



Contiguous memory management schemes:

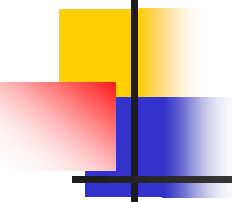
➤ Single Partition

In this scheme, the main memory is divided into two contiguous areas or partitions. The operating systems reside permanently in one partition, generally at the lower memory, and the user process is loaded into the other partition.

➤ Multiple Partition

The operating system divides the available main memory into multiple parts to load multiple processes into the main memory.

- ✓ **Fixed Partitioning:** Each partition or block is of fixed size and can hold single process.
- ✓ **Dynamic Partitioning:** In this scheme the partitions used are of variable size. Each process occupies only as much memory as they require when loaded for processing.



A process can be allocated with a free memory partition or block based three strategies:

1. First fit:

This type will allocate the first free block that is big enough to satisfy the memory request of a process from the beginning of the memory.

2. Best fit:

This type will allocate the smallest free block that is big enough to satisfy the memory request of a process by searching the whole memory.

3. Worst fit:

This will allocates the process to the partition, which is the largest sufficient freely available partition in the main memory.

Fragmentation

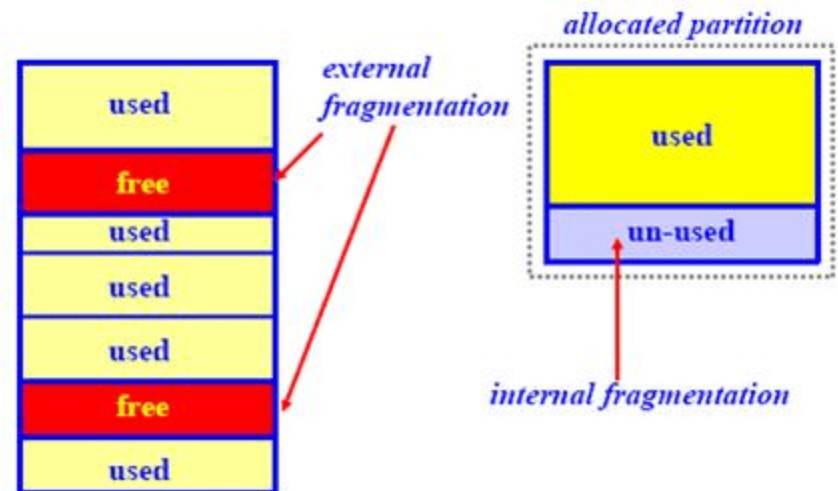
Fragmentation is an unwanted memory space problem in the operating system occurred when the processes are loaded and unloaded from memory.

➤ Internal Fragmentation

When a process with smaller amount of memory request is allocated with a larger memory block, then a free space is created in the given memory block that becomes unused. This condition is called as internal fragmentation.

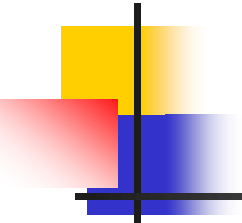
➤ External Fragmentation

External fragmentation occurs when there is enough memory space to complete a memory request of some process, but it is not contiguous.



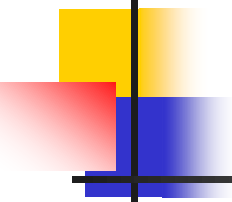


S.NO.	Contiguous Memory Allocation	Non-contiguous Memory Allocation
1	Contiguous memory allocation allocates consecutive blocks of memory to a file/process.	Non-Contiguous memory allocation allocates separate blocks of memory to a file/process.
2	Faster in Execution.	Slower in Execution.
3	It is easier for the OS to control.	It is difficult for the OS to control.
4	Overhead is minimum as not much address translations are there while executing a process.	More Overheads are there as there are more address translations.
5	Internal fragmentation occurs in Contiguous memory allocation method.	External fragmentation occurs in Non-Contiguous memory allocation method.

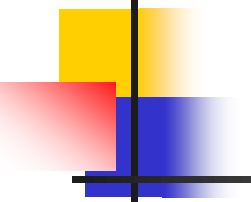


6	It includes single partition allocation and multi-partition allocation.	It includes paging and segmentation.
7	Wastage of memory is there.	No memory wastage is there.
8	In contiguous memory allocation, swapped-in processes are arranged in the originally allocated space.	In non-contiguous memory allocation, swapped-in processes can be arranged in any place in the memory.

Compaction: It is process of grouping or combining all the fragmented memory blocks into a single large free block.



S.NO	Internal Fragmentation	External Fragmentation
1	It occurs when free space within a memory block becomes unused.	It occurs when non contiguous free blocks becomes unused due to dynamic allocation.
2	It happens when the method or process is larger than the memory.	It happens when the method or process is removed from memory.
3	The solution of internal fragmentation is best-fit block.	Solution of external fragmentation is compaction, paging and segmentation.
4	Internal fragmentation occurs when memory is divided into fixed sized partitions.	External fragmentation occurs when memory is divided into variable size partitions based on the size of processes.



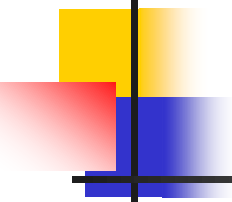
□ Paging

- **Paging** is another memory-management scheme that also permits non-contiguous physical address space.
- In this the main memory is divided into equal size partitions.
- The partitions of the secondary memory are called **pages** while the partitions of the main memory are called **frames**.
- They are divided into equal size partitions to have maximum utilization of the main memory and hence avoid external fragmentation.

➤ Translation of logical Address into physical Address

The logical address consists of two parts page number and page offset.

1. **Page Number:** It tells the exact page of the process which the CPU wants to access.
2. **Page Offset:** It tells the exact word on that page which the CPU wants to read.



Physical Address: The physical address consists of two parts frame number and page offset.

1.Frame Number: It tells the exact frame where the page is stored in physical memory.

2. Page Offset: It tells the exact word on that page which the CPU wants to read. It requires no translation as the page size is the same as the frame size so the place of the word which CPU wants access will not change.

Page table: A page table contains the frame number corresponding to the page number of some specific process. So, each process will have its own page table. A register called Page Table Base Register (PTBR) which holds the base value of the page table.

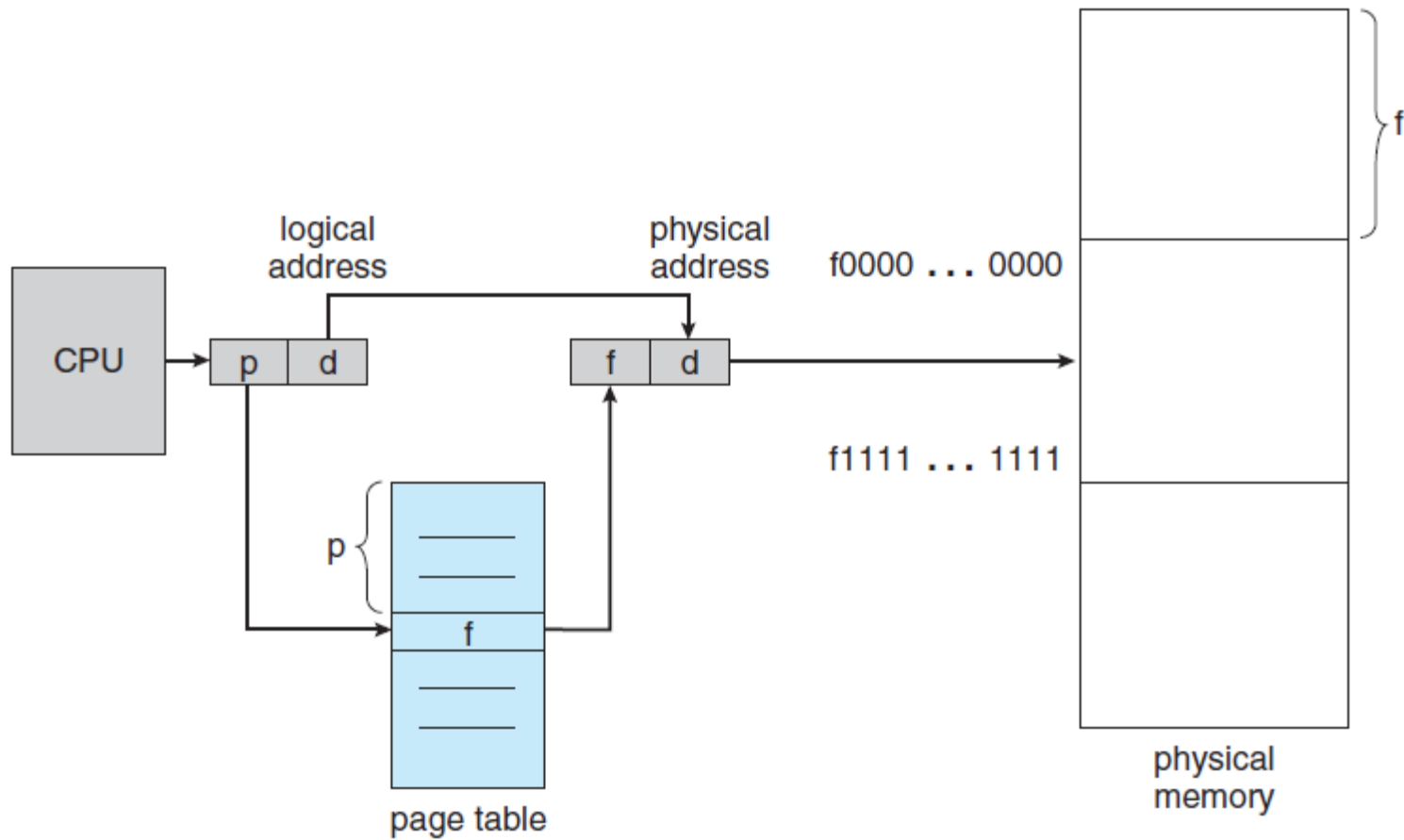
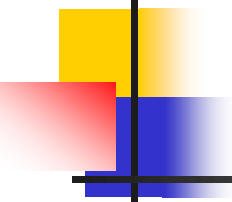


Figure 8.10 Paging hardware.

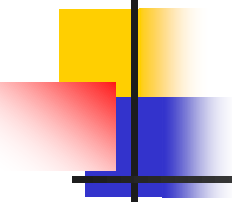


Advantages of Paging

1. There is no external fragmentation as it allows us to store the data in a non-contiguous way.
2. Swapping is easy between equal-sized pages and frames.

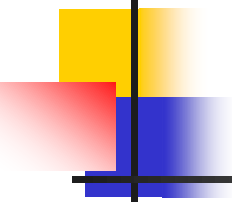
Disadvantages of Paging

1. As the size of the frame is fixed, so it may suffer from internal fragmentation.
2. The access time increases because of paging as the main memory has to be now accessed two times.
3. For every process, we have an independent page table and maintaining the page table is extra overhead.

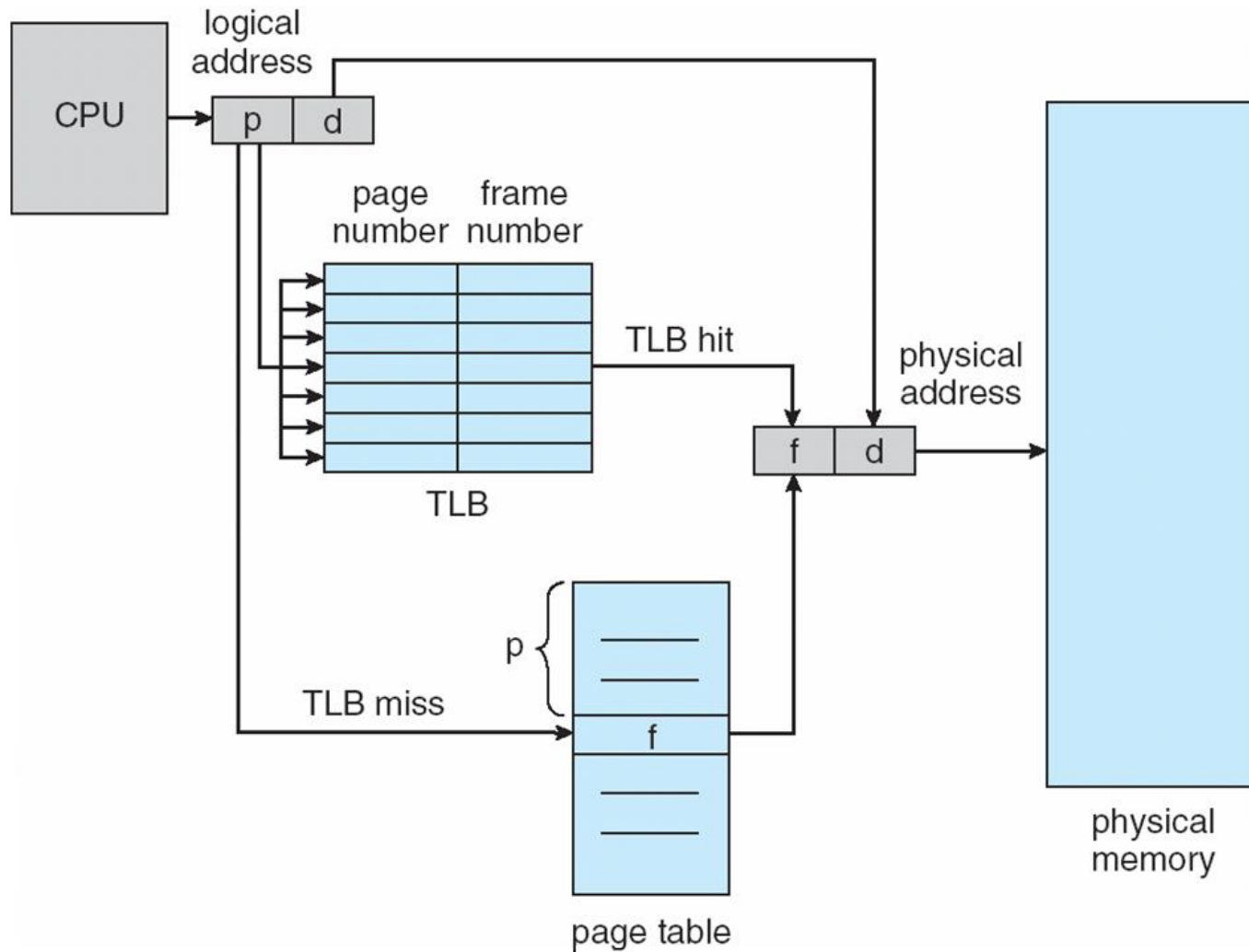


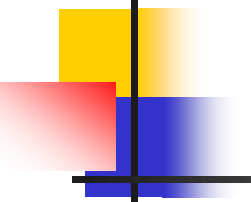
Paging with Translation Look-aside Buffers (TLBs)

- Since Page table is kept in main memory it can be accessed by the pointer **Page-table base register** (PTBR)
- **Page-table length register** (PTLR) indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory or translation look-aside buffers (TLBs)**
- Each TLB entry – uniquely identifies each process to provide address-space protection for that process.
- TLBs are typically small size compared to Page Table.

- 
- Each time when address translation need to be done, the page reference will be checked in the TLB first.
 - If it present then it is called **TLB Hit**, then its frame number will be passed. If it is not present in the table then it called **TLB Miss**.
 - On a TLB miss, then page will be searched in the Page Table and later value is loaded into the TLB for faster access next time.
 - Paging hardware with TLB consists of both TLB table and Page Table.

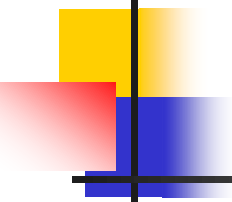
Paging Hardware With TLB





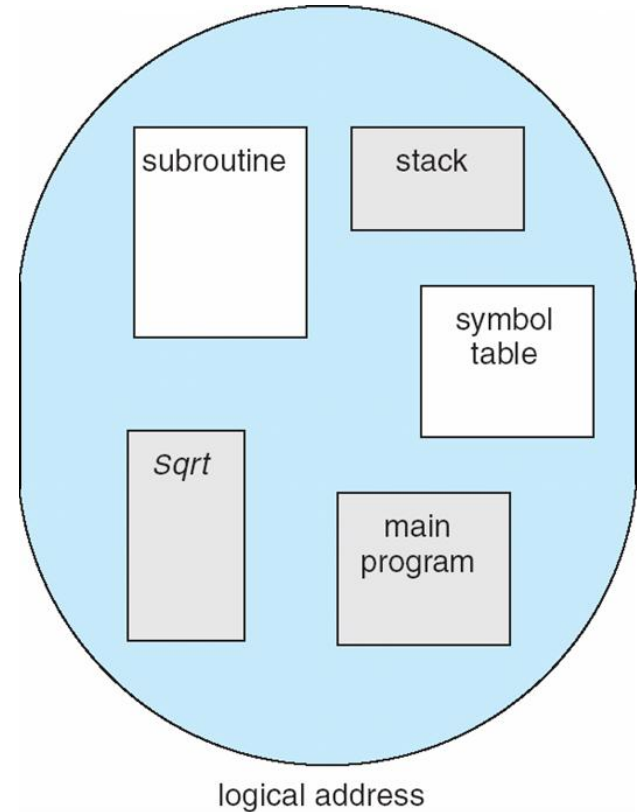
❑ Segmentation

- **Segmentation** is a memory-management scheme that supports programmer view of memory and the logical address space is divided into collection of segments.
- Segmentation permits the physical address space of a process to be noncontiguous.
- Each segment has a name and a length. The addresses specify both the segment name and the offset within the segment.
- For simple implementation segments are numbered and are referred to by a segment number, rather than by a segment name.
- Thus, a logical address consists of a two tuple:
 $\langle \text{segment-number}, \text{offset} \rangle$.

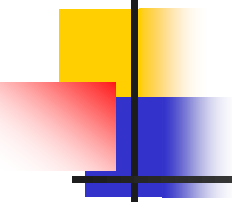


Compiler might create separate segments for the following:

1. The code
2. Global variables
3. The heap, from which memory is allocated
4. The stacks used by each thread
5. The standard C library



User's View of a Program



Segmentation Hardware:

- It is used to map two-dimensional user-defined addresses into one-dimensional physical address with the help of segment table.
- Each entry in the segment table has a segment base and a segment limit.
- The segment base contains the starting physical address where the segment resides in memory, and the segment limit specifies the length of the segment.
- Address generated by the CPU is divided into:
 1. Segment number (s): Number of bits required to represent the segment.
 2. Segment offset (d): Number of bits required to represent the size of the segment.

In segmentation, the logical address generated by the CPU has two parts:

segment number

offset within that segment

So an address looks like: (segment number, offset)

How it works

Each segment has an entry in a segment table.

The segment table stores:

base address = starting physical address of the segment

limit = size of the segment

When a logical address is given:

1.OS checks the segment number.

2. It finds the base and limit from the segment table.

3.It checks whether the offset is within the segment size.

4. If valid, physical address is:

Physical Address = Base + Offset

Example

Suppose segment table is:

Segment 0: base = 1000, limit = 400

Segment 1: base = 3000, limit = 500

Now logical address = (1, 120)

For segment 1:

base = 3000

offset = 120

So:

Physical address = $3000 + 120 = 3120$

If the offset were 600, it would be invalid because it exceeds the limit 500.

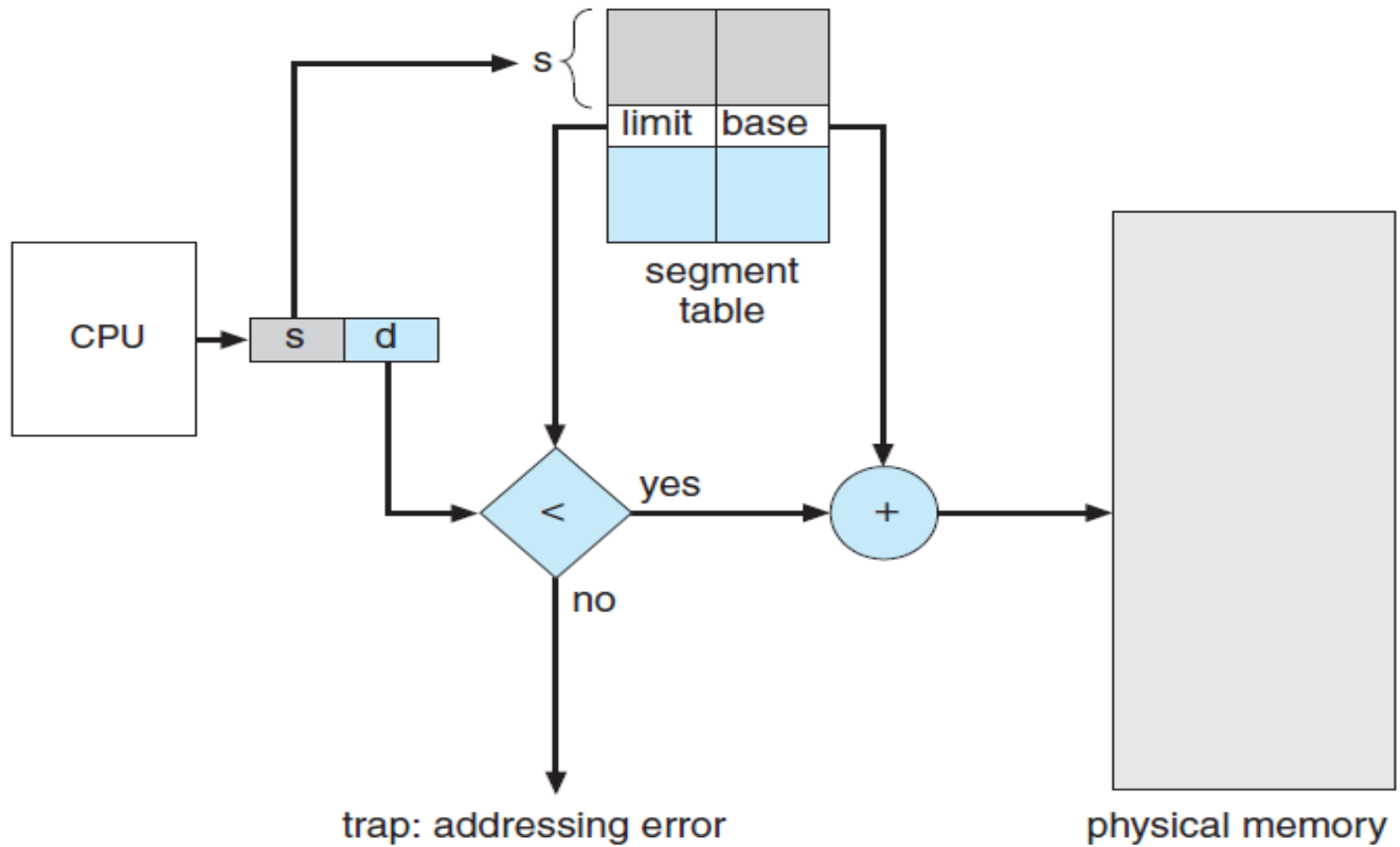
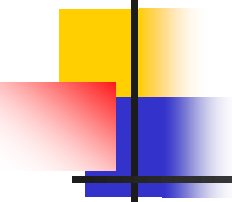


Figure 8.8 Segmentation hardware.



Advantages of Segmentation –

- No Internal fragmentation.
- Segment Table consumes less space in comparison to Page table in paging.

Disadvantage of Segmentation –

- As processes are loaded and removed from the memory, the free memory space is broken into little pieces, causing External fragmentation.

Segmentation is useful because it matches the programmer's view of memory.

A program is naturally written in parts like code, data, and stack, so segmentation stores memory in that same logical way.

Advantages

- **reflects logical structure of a program**
- **easy protection for different segments**
- **easy sharing of specific segments like code**
- **supports modular programming**

Disadvantages

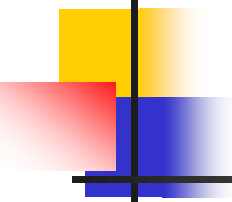
- **causes external fragmentation**
- **memory management becomes complex**
- **compaction may be needed**

Difference from paging

- **Paging divides memory into fixed-size pages**
- **Segmentation divides memory into variable-size logical segments**

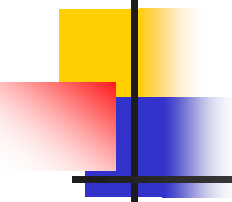
A simple way to remember:

- **Paging = physical division**
- **Segmentation = logical division**



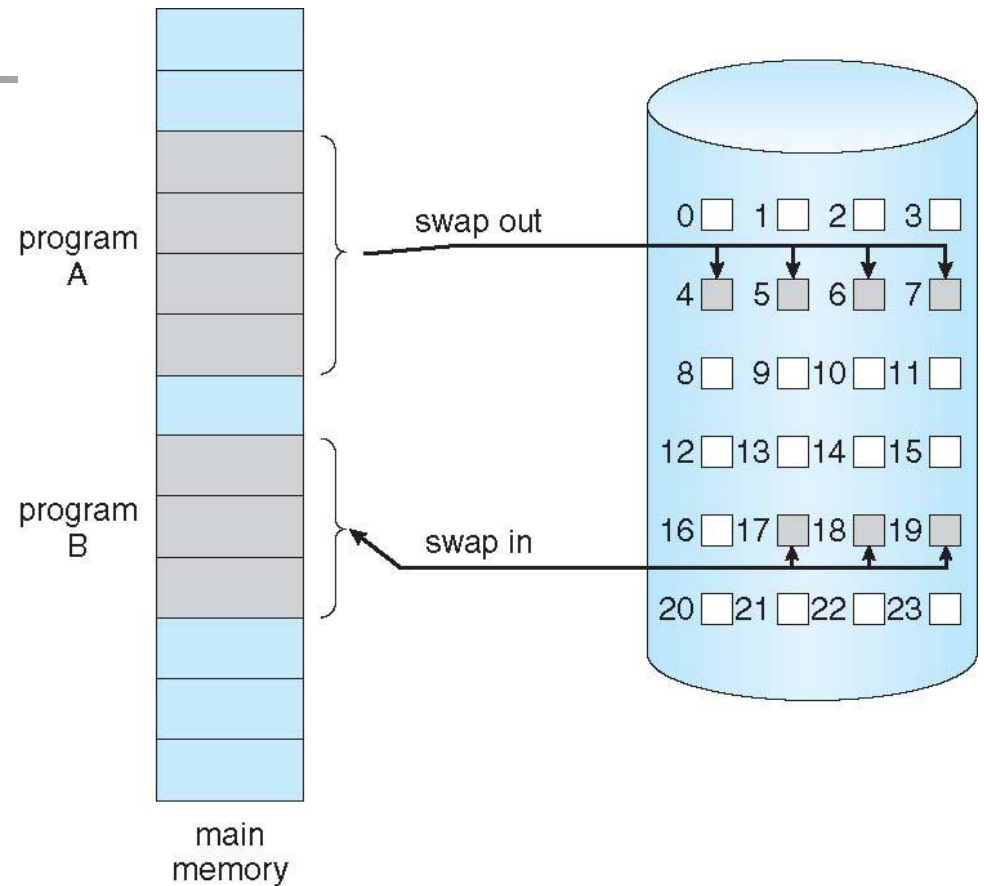
❑ Virtual Memory

- Virtual Memory is a storage mechanism which offers user an illusion of having a very big main memory.
- It is done by treating a part of secondary memory as the main memory.
- In Virtual memory, the user can store processes with a bigger size than the available main memory.
- Instead of loading one long process in the main memory, the OS loads the various parts of more than one process in the main memory.
- Virtual memory is mostly implemented with demand paging and demand segmentation.

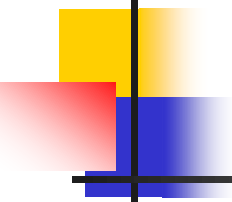


□ Demand Paging

- Demand paging is a memory management technique used by operating systems to optimize the use of memory resources.
- In demand paging, we only load the required pages of a process into the main memory instead of loading the entire process.
- A demand paging mechanism is very much similar to a paging system with swapping where processes stored in the secondary memory and pages are loaded only on demand, not in advance.
- During the program execution, if the program references a page that may not be available in the main memory then this scheme will bring that page on demand from the secondary storage.

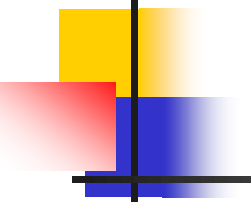


- It uses a **Lazy swapper** – never swaps a page into memory unless page will be needed.
- Swapper that deals with pages is called **Pager**.



Advantages of Demand Paging:

- Less I/O needed, no unnecessary I/O
- Less memory needed
- Faster response
- More users can be use memory at the same time.



Page Fault

If there is a reference to a page, first reference to that page will trap to operating system:

Steps to follow during page fault:

1. Operating system looks at another table to decide:

Invalid reference $\Rightarrow$ abort

Just not in memory

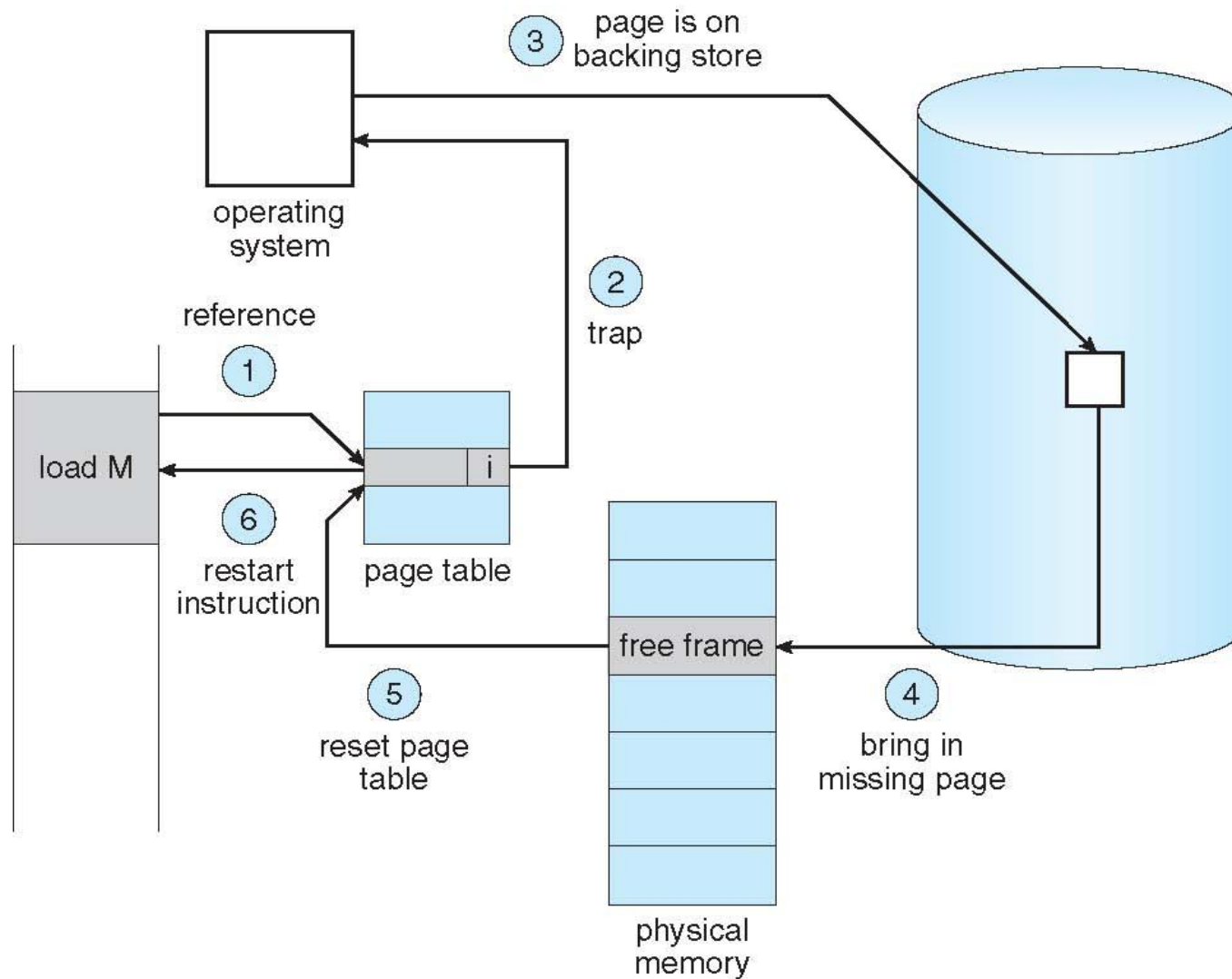
2. Find free frame

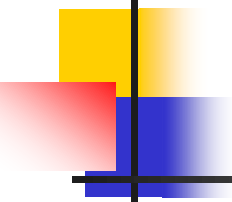
3. Swap page into frame via scheduled disk operation

4. Reset tables to indicate page now in memory

Set validation bit = v

5. Restart the instruction that caused the page fault



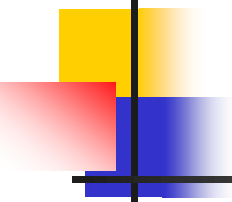


Thrashing

Thrashing is a condition or a situation where the operating system is spending a major portion of its time in servicing the page faults, rather than actual processing.

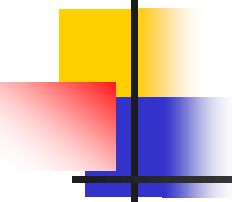
The effect of Thrashing is:

- CPU becomes idle.
- Decreasing the utilization increases the degree of multiprogramming and hence bringing more processes at a time which in fact increases the thrashing exponentially.



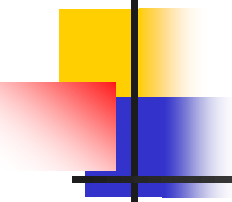
❑ Page Replacement Algorithm

- Page Replacement Algorithm decides which page to remove, also called swap out when a new page needs to be loaded into the main memory.
- Page Replacement happens when a requested page is not present in the main memory and the available space is not sufficient for allocation to the requested page.
- A page replacement algorithm tries to select which pages should be replaced so as to minimize the total number of page misses.
- If a process requests for page and that page is found in the main memory then it is called **page hit**.
- If the requested page is not found in the main memory then it is called as **page miss** or **page fault**.



Some Page Replacement Algorithms :

- First In First Out (FIFO)
- Least Recently Used (LRU)
- Optimal Page Replacement



First In First Out (FIFO)

- This is the simplest page replacement algorithm. In this algorithm, the OS maintains a queue that keeps track of all the pages in memory, with the oldest page at the front and the most recent page at the back.
- When there is a need for page replacement, the FIFO algorithm, swaps out the page at the front of the queue, that is the page which has been in the memory for the longest time.

Advantages

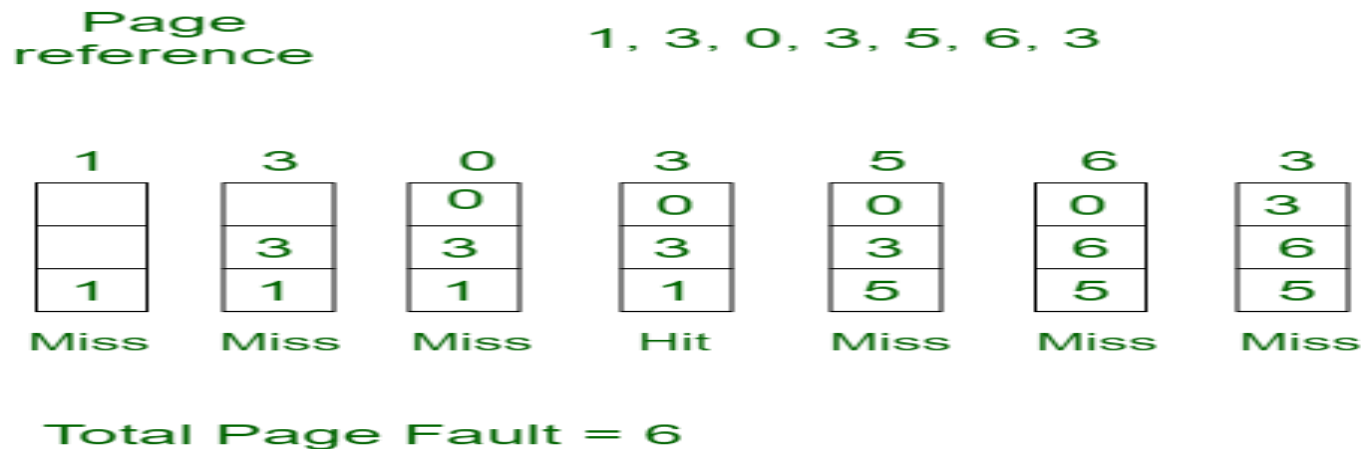
- Simple and easy to implement.
- Low overhead.

Disadvantages

- Poor performance.
- Suffers from Belady's Anomaly(i.e. more page faults when we increase the number of page frames).

For Example:

Consider page reference string 1, 3, 0, 3, 5, 6 with 3 page frames. Find number of page faults.



Belady's anomaly – Belady's anomaly proves that it is possible to have more page faults when increasing the number of page frames while using the First in First Out (FIFO) page replacement algorithm.

Optimal Algorithm

- Replace page that will not be used for longest period of time
- OS needs future knowledge of page references in advance.
- Need to sort according to their next usage/ reference
- Overhead for OS & complex to implement.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2	2		2		2							7		
	0	0	0		0	4		0		0							0		
		1	1		3	3		3		1							1		

page frames

Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

page frames

- 12 faults – better than FIFO but worse than OPT